Exact Smooth-Rotation Estimation in Near-Linear Time for ARAP Surface Deformation

Mauricio Cele Lopez Belon

Abstract. As-Rigid-As-Possible deformation with Smooth Rotations (SR-ARAP [5]) has recently gained popularity in applications that need an isometric surface mapping, because the quality of its results is comparable to that of more costly volumetric techniques operating on tetrahedral meshes. SR-ARAP minimizes a non-linear least-squares energy with a local/global optimization. Its local step estimates the rotation of each surface cell together with the rotations of its neighbors, so the rotations are codependent across the whole surface. The local step is currently solved by relaxation, and since only a few relaxation iterations can be afforded per step, the rotations it produces are approximate. We show that, with linearized quaternion rotations, the local step can be solved exactly. After a diagonal scaling it becomes a single sparse, symmetric positive definite linear system, which we prove to be the exact stationarity condition of a well-defined quadratic energy. A feedback term towards the previous rotations bounds the system's smallest eigenvalue independently of the mesh, which keeps it well conditioned. We prove that preconditioning the system with a Cholesky factorization of its position-independent part, which is factored once per set of constraints, bounds the condition number by a constant that shrinks as the mesh is refined. Conjugate gradients then reach the exact rotations, to 10^{-12} rad, in 5 to 20 iterations of linear cost, so the exact local step scales nearly linearly with the mesh and is up to 49 times faster than a direct factorization on large meshes. Compared with the original SR-ARAP local step, the converged shapes have 5–12% lower ARAP distortion and smoother rotations, and they reach the quality at which SR-ARAP stops 1.4 to 4.6 times sooner. Combined with a multiresolution scheme that transfers the rotations by harmonic interpolation, the method deforms a 173,000-vertex model interactively, with the exact local steps taking 2.4 ms of a 29 ms frame.

Mathematics Subject Classification (2010). Parallel algorithms 68W10; Clifford algebras, spinors 15A66.

Keywords. Geometric Algebra, Quaternion Estimation, Mesh Deformation.



(A) Rest (B) Reaching (C) Victory (D) Kick

FIGURE 1. The Armadillo (172,974 vertices) posed with six handles (orange spheres). Rotations are solved in closed form on a 502-vertex control mesh and transferred to the full surface by harmonic interpolation; a complete interactive frame takes 29 ms on a laptop CPU, of which its three exact local steps take 2.4 ms.

1. Introduction

High-quality surface deformation aims to preserve the local properties of a shape as much as possible, removing distortion in the form of shear and stretch. As-Rigid-As-Possible deformation with Smooth Rotations (SR-ARAP) [5] creates a surface mapping that minimizes the deviation from rigid behavior at the local scale. It also makes the rigid transformations vary smoothly over local neighborhoods, distributing distortion uniformly over the surface. The resulting deformations preserve the shape not only at small scales but also at large scales, which significantly increases the quality of the final result.

The current SR-ARAP method has two main disadvantages. The first is the local step, which looks for the best rotation of each surface cell while keeping neighboring rotations similar to each other. It needs to converge globally over the whole mesh. Currently the local step is solved with a relaxation method: the rotation of each cell is optimized while its neighbors' rotations are kept fixed, and the process is repeated until convergence. Although the cost of each relaxation iteration can be reduced considerably by avoiding SVD computations [11], a fixed number of iterations is usually performed in order to bound the solver time. That compromises the quality of the results, since a fixed number of iterations (usually two or three) is not enough to reach optimal rotations across the surface. The second disadvantage is the high computational cost, which increases drastically with the number of vertices: SR-ARAP is a non-linear technique that requires several local/global iterations to converge.

In this paper we address both drawbacks. First, we solve the local step as a single linear system of equations, avoiding the need to solve a series of codependent SVD problems iteratively. We show that, after a diagonal scaling, this system is symmetric positive definite and is the exact stationarity condition of a well-defined quadratic energy (Section 5.6). We bound its condition number and show that a preconditioner built from its position-independent part, factored once per set of constraints, makes conjugate gradients converge in a number of iterations that does not grow with the mesh (Section 5.8); the exact local step therefore costs nearly linear time, and on the meshes we tested it reaches the quality of the original SR-ARAP sooner. Second, we propose a simple multiresolution method: the non-linear problem is first solved on a simplified surface mesh, and the optimized rotations are then transferred to the full-resolution mesh by harmonic interpolation, reusing the Laplacian matrix of the global step.

2. Related Work

Since the focus of this paper is As-Rigid-As-Possible (ARAP) surface deformation, we only review previous work on ARAP deformation. ARAP surface modeling was introduced by Sorkine and Alexa [10] to produce robust and physically plausible deformations by minimizing the deviation from rigidity of the local transformation associated with every vertex. Although results on low-resolution meshes look close to physical deformations, the method does not achieve satisfactory results on high-resolution meshes. The reason is that the ARAP energy is minimized by preserving rigidity on large parts of the mesh at the expense of concentrating all the distortion on small parts of it. This problem was solved by the Smooth Rotation enhanced ARAP method (SR-ARAP) of Levi and Gotsman [5], in which the deformation energy is extended with a term that penalizes differing rotations of nearby vertices, significantly increasing the quality of the final result. Chao *et al.* [1] derive the ARAP energy from the elastic energy of continuum mechanics, showing a consistent discretization for volumetric ARAP with tetrahedral cells and for 2D ARAP with triangular cells. However, the ARAP energy does not have a consistent discretization for triangulated surfaces in 3D. SR-ARAP [5] also addresses this issue, providing an energy with a consistent discretization for surfaces in 3D. Further enhancements to SR-ARAP have addressed performance on large meshes [8, 11], made the physical stiffness adjustable [2, 4], and introduced local scaling to the rotations, turning rigid transformations into similarity transformations [3].

3. As-Rigid-As-Possible Shape Deformation with Smooth Rotations

Let Q be the rest mesh, with vertices $q_1, \ldots, q_m$, and P the deformed mesh, with vertices $p_1, \ldots, p_m$ and the same connectivity. Denote by $N(i)$ the set

of 1-ring neighbors of vertex i, by $d_i = |N(i)|$ its degree (valence), and by $q_{ij} = q_j - q_i$ and $p_{ij} = p_j - p_i$ the directed edges. The discrete As-Rigid-As-Possible energy with Smooth Rotations (SR-ARAP) is

$$E(P,R) = \sum_{i=1}^{m} \Big(\sum_{j \in N(i)} c_{ij} \, \| R_i \, q_{ij} \, \tilde{R}_i - p_{ij} \|^2 + \alpha A \sum_{j \in N(i)} w_{ij} \, \| R_j - R_i \|^2 \Big), \tag{3.1}$$

where $R_1, \ldots, R_m \in \mathbb{H}$ are unit quaternions, $\tilde{R}_i$ is the conjugate of R_i, and edge vectors are identified with pure quaternions. The membrane weights $c_{ij} = \max\big(0, \frac{1}{2}(\cot \alpha_{ij} + \cot \beta_{ij})\big)$ are the familiar cotangent weights, where α_{ij} and β_{ij} are the angles opposite the edge ij. They are symmetric, $c_{ij} = c_{ji}$, and non-negative, but not normalized. The bending weights w_{ij} are positive and normalized per vertex, $\sum_{j \in N(i)} w_{ij} = 1$; in this paper $w_{ij} = 1/d_i$. Since d_i varies from vertex to vertex, they are *not* symmetric in general: $w_{ij} \neq w_{ji}$. A is the mesh area, which makes the energy scale invariant, and α is a positive scalar parameter.

The main idea of the method is to break the surface into overlapping cells and keep the transformation of each cell as rigid as possible in the least-squares sense. The cells must overlap to avoid stretching or shearing the surface at cell boundaries. The first term is a membrane energy, which penalizes stretching and shearing of a cell; the second term is a bending energy, which penalizes the difference between a cell's rotation and the rotations of its neighboring cells. The membrane term lowers the distortion of a cell by keeping the differential of the map close to rigid. The bending term keeps the variation of the rotations within a cell's neighborhood low, so that the neighborhood transforms as a unit as much as possible.

The vertices of Q are in their original positions, the vertices of P are the deformed positions, and the quaternion R_i is the best rigid transformation, in the least-squares sense, relating the original and the deformed cell. This is a non-linear optimization problem, typically solved by an iterative local/global method that solves two linear sub-problems in each iteration. The first, the *local step*, keeps the vertices of P constant and finds the best rotation R_i for each cell. The second, the *global step*, keeps the rotations R_i constant and computes the optimal deformed vertices p_i in the least-squares sense.

4. Global Step

The global step computes the optimal vertices $p_1, \ldots, p_m$ for fixed rotations:

$$\min_{p_1,\ldots,p_m \in \mathbb{R}^3} \sum_{i=1}^{m} \sum_{j \in N(i)} c_{ij} \, \| R_i \, q_{ij} \, \tilde{R}_i - p_{ij} \|^2.$$

The bending term of (3.1) does not depend on the positions, so it is omitted. Vertex p_i appears in the edges p_{ij} of its own cell and in the edges $p_{ji} = -p_{ij}$ of its neighbors' cells. Since $c_{ij} = c_{ji}$ and $q_{ji} = -q_{ij}$, setting the partial

derivative with respect to p_i to zero gives the linear system of Equation (4.1) [6]:

$$\sum_{j \in N(i)} c_{ij}\,(p_j - p_i) = \sum_{j \in N(i)} \frac{c_{ij}}{2}\left(R_i\, q_{ij}\, \tilde{R}_i + R_j\, q_{ij}\, \tilde{R}_j\right), \tag{4.1}$$

which can be written in matrix form as $\mathrm{L}\,\mathrm{P} = \mathrm{C}$. Here L is the $m \times m$ discrete Laplace–Beltrami operator, $(\mathrm{L}\,\mathrm{P})_i = \sum_{j \in N(i)} c_{ij}(p_j - p_i)$, P is the $m \times 3$ matrix of target positions, and C is the $m \times 3$ matrix whose ith row is the right-hand side of Equation (4.1). The operator L is symmetric and negative semi-definite, since

$$-x^T \mathrm{L}\, x = \sum_{\{i,j\} \in \mathcal{E}} c_{ij}\,(x_i - x_j)^2 \geq 0 \quad \text{for all } x \in \mathbb{R}^m,$$

where $\mathcal{E}$ is the set of mesh edges, each counted once.

4.1. Positional constraints

Constraints of the form $p_i = p_i^{const}$ for i in a set $\mathcal{C}$ of constrained vertices are incorporated by substituting the corresponding variables. Let $\mathcal{F}$ be the remaining (free) vertices. Partitioning L, P and C accordingly, the rows of $\mathcal{F}$ read $\mathrm{L}_{\mathcal{F}\mathcal{F}}\mathrm{P}_{\mathcal{F}} + \mathrm{L}_{\mathcal{F}\mathcal{C}}\mathrm{P}_{\mathcal{C}} = \mathrm{C}_{\mathcal{F}}$, that is,

$$(-\mathrm{L}_{\mathcal{F}\mathcal{F}})\,\mathrm{P}_{\mathcal{F}} = -\mathrm{C}_{\mathcal{F}} + \mathrm{L}_{\mathcal{F}\mathcal{C}}\,\mathrm{P}_{\mathcal{C}}^{const}. \tag{4.2}$$

This system is square, so no least-squares formulation (normal equations $\mathrm{L}^T\mathrm{L}$) is needed; forming the normal equations would square the condition number and increase the fill-in of the factorization.

Proposition 4.1. *If every connected component of the mesh, connected through edges with $c_{ij} > 0$, contains a constrained vertex, then $-\mathrm{L}_{\mathcal{F}\mathcal{F}}$ is symmetric positive definite.*

Proof. $-\mathrm{L}_{\mathcal{F}\mathcal{F}}$ is a principal submatrix of the symmetric matrix $-\mathrm{L}$, so it is symmetric. For $x \in \mathbb{R}^{|\mathcal{F}|}$, extend it by zero on $\mathcal{C}$; then $x^T(-\mathrm{L}_{\mathcal{F}\mathcal{F}})x = \sum_{\{i,j\} \in \mathcal{E}} c_{ij}(x_i - x_j)^2 \geq 0$. Equality holds only if x is constant on every component, and since every component contains a constrained vertex, where the extended x is zero, this forces $x = 0$. □

Equation (4.2) is therefore solved with a sparse Cholesky factorization of $-\mathrm{L}_{\mathcal{F}\mathcal{F}}$, which depends only on the mesh and the constraint set. It is computed once; every global step then costs two sparse triangular solves per coordinate. The three coordinates share the factorization.

5. Local Step

As described in the previous sections, the rotations of the cells are codependent over the whole mesh. The current approach optimizes them with a relaxation method: the rotation of each cell is computed independently while the neighboring rotations are kept fixed, and this is repeated until global convergence. At least two relaxation iterations must be performed in each

global iteration. In this section we show how the local step can be solved in closed form, as a single sparse linear system, eliminating relaxation iterations entirely.

5.1. Linearized membrane energy

For a unit quaternion R_i, $\|R_i\, q_{ij}\, \tilde{R}_i - p_{ij}\| = \|R_i\, q_{ij} - p_{ij}\, R_i\|$, since right multiplication by the unit quaternion R_i preserves the norm. The expression $R_i\, q_{ij} - p_{ij}\, R_i$ is linear in the four coefficients of R_i, $R_i\, q_{ij} - p_{ij}\, R_i = M_{ij} R_i$, so the membrane term of cell i is the quadratic form

$$\sum_{j\in N(i)} c_{ij}\, \|R_i\, q_{ij} - p_{ij}\, R_i\|^2 = R_i^T\, M_i\, R_i, \qquad M_i = \sum_{j\in N(i)} c_{ij}\, M_{ij}^T M_{ij}, \quad (5.1)$$

where M_i is a symmetric positive semi-definite 4×4 matrix built from q_{ij} and p_{ij} (see Section 7). In what follows quaternions are treated as vectors of $\mathbb{R}^4$ and the unit-norm constraint is dropped; the solutions are normalized afterwards. As shown in [11], this linearization gives an accurate approximation of the optimal quaternion in the least-squares sense, and our results confirm that it does not affect the accuracy of SR-ARAP rotations in any significant way.

The linearized energy of the local step is then

$$E(R) = \sum_{i=1}^{m} \Big(R_i^T M_i\, R_i + \alpha A \sum_{j\in N(i)} w_{ij}\, \|R_j - R_i\|^2 \Big). \quad (5.2)$$

5.2. The relaxation fixed point

Relaxation minimizes each cell's energy

$$E_i(R_i) = R_i^T M_i\, R_i + \alpha A \sum_{j\in N(i)} w_{ij}\, \|R_j - R_i\|^2$$

with the neighboring rotations R_j fixed. Using $\sum_j w_{ij} = 1$,

$$\tfrac{1}{2}\,\frac{\partial E_i}{\partial R_i} = M_i\, R_i + \alpha A \sum_{j\in N(i)} w_{ij}\, (R_i - R_j) = (M_i + \alpha A\, I)\, R_i - \alpha A \sum_{j\in N(i)} w_{ij}\, R_j,$$

and setting it to zero gives

$$(M_i + \alpha A\, I)\, R_i = \alpha A \sum_{j\in N(i)} w_{ij}\, R_j. \quad (5.3)$$

Relaxation converges, when it does, to a set of rotations satisfying (5.3) at every vertex simultaneously. These $4m$ equations are linear, so their common solution can be computed directly:

$$(\mathrm{M} - \mathrm{W})\, \mathrm{R} = 0, \quad (5.4)$$

where R is the $4m \times 1$ column vector stacking the coefficients of the quaternions R_i, M is the $4m \times 4m$ block-diagonal matrix with blocks $M_i + \alpha A\, I$, and W is the $4m \times 4m$ matrix whose 4×4 block (i, j) is $\alpha A\, w_{ij}\, I$ for $j \in N(i)$, and zero otherwise.

M is symmetric, but W is not: block (i, j) is $\alpha A\, w_{ij}\, I$ while block (j, i) is $\alpha A\, w_{ji}\, I$, and with per-vertex normalized weights $w_{ij} \neq w_{ji}$ whenever $d_i \neq d_j$. System (5.4) is therefore not symmetric as it stands.

5.3. The true gradient of E

Equation (5.3) is the stationarity condition of each cell energy E_i with respect to its own rotation, but it is not the stationarity condition of the total energy (5.2). The rotation R_i also appears in the bending terms of its neighbors' cells, so

$$\tfrac{1}{2}\,\frac{\partial E}{\partial R_i} = M_i\, R_i + \alpha A \sum_{j \in N(i)} (w_{ij} + w_{ji})\,(R_i - R_j). \tag{5.5}$$

The minimizer of E satisfies a symmetric system with the symmetrized weights $w_{ij} + w_{ji}$; it coincides with (5.3) only when $w_{ij} = w_{ji}$, which per-vertex normalized weights do not satisfy. The closed-form solution of (5.4), like the fixed point of relaxation, is therefore not the minimizer of (5.2). In Section 5.6 we identify the energy it does minimize exactly.

5.4. Rotation constraints R_i^{const}

The vertices inside the handles are constrained, $i \in \mathcal{C}$. Their rotations are not free variables of the local step: they are computed directly from Equation (5.3), with the neighboring rotations R_j^{prev} known from the previous iteration,

$$R_i^{const} = (M_i + \alpha A\, I)^{-1}\, \frac{\alpha A}{d_i} \sum_{j \in N(i)} R_j^{prev}, \qquad i \in \mathcal{C}, \tag{5.6}$$

and normalized. Computing each constraint quaternion therefore amounts to solving a small 4×4 linear system, which is always possible since $M_i + \alpha A\, I$ is positive definite.

5.5. Rotation feedback

The system (5.4) finds quaternions from scratch: it does not take into account their previous state, that is, their temporal coherence. That can lead to undesired results in an animation sequence. In particular, the sense of the rotations found in one animation step does not always match the sense of the rotations of the previous step, and the abrupt change causes animation artifacts, especially for large rotations.

The previous rotation of each cell can be added to its energy as a *feedback* term, which acts as a *hint* towards quaternions close to the ones of the previous iteration. Given R_i^{prev} and a small positive scalar e, we make R_i a neighbor of itself: the cell energy becomes

$$E_i(R_i) = R_i^T\, M_i\, R_i + \alpha A \sum_{j \in N(i)} w_{ij}\, \|R_j - R_i\|^2 + \alpha A\, \epsilon_i\, \|R_i - R_i^{prev}\|^2,$$

with

$$w_{ij} = \frac{1}{d_i + e}, \qquad \epsilon_i = \frac{e}{d_i + e},$$

so that $\sum_{j\in N(i)} w_{ij} + \epsilon_i = 1$. The strength of the feedback is given by e. Its stationarity condition replaces (5.3):

$$\begin{aligned}(M_i + \alpha A\, I)\, R_i &= \alpha A \sum_{j\in N(i)} w_{ij}\, R_j + \alpha A\, \epsilon_i\, R_i^{prev} \\ &= \frac{\alpha A}{d_i + e}\Big(\sum_{j\in N(i)} R_j + e\, R_i^{prev}\Big),\end{aligned} \tag{5.7}$$

and the global system becomes

$$(\mathrm{M} - \mathrm{W})\,\mathrm{R} = \alpha A\, \mathrm{E}\, \mathrm{R}^{prev}, \qquad \text{s.t. } R_i = R_i^{const} \text{ for } i \in \mathcal{C}, \tag{5.8}$$

where E is the block-diagonal matrix with blocks $\epsilon_i I$ and R^{prev} is the $4m \times 1$ column vector stacking the quaternions R_i^{prev} of the previous iteration.

Benefits and cost of the feedback. The feedback term has four benefits. (i) *Temporal coherence*: during interactive editing only a few iterations run per frame, and the feedback keeps every local step close to the previous one, so rotations do not pop between frames. (ii) *Large rotations*: near and beyond 180° the linearized fit has two nearly equivalent senses of rotation; anchoring each cell to its previous rotation favors the branch it was following, which prevents the abrupt changes in the sense of rotation described above and lets a large twist spread evenly along the surface as it grows (Section 9.6). (iii) *Conditioning*: it bounds the smallest eigenvalue of the rotation system by e for any mesh and any constraint set (Proposition 5.4), so the system stays solvable even without constraints, and iterative solvers of it converge in a number of iterations independent of the mesh (Section 9). (iv) *Little bias*: it barely changes the converged shape (Remark 5.1). Its cost is a damping of the outer local/global loop, which needs more iterations to reach its fixed point when the rotations are large.

Remark 5.1. At a fixed point of the local/global loop the rotations no longer change, $R_i = R_i^{prev}$, so (5.7) becomes $(M_i + \alpha A\, I)R_i = \frac{\alpha A}{d_i+e}\big(\sum_j R_j + eR_i\big)$, that is,

$$\Big(\frac{d_i + e}{d_i}\, M_i + \alpha A\, I\Big) R_i = \frac{\alpha A}{d_i} \sum_{j\in N(i)} R_j.$$

This is the fixed point without feedback, (5.3) with $w_{ij} = 1/d_i$, except that the membrane term of each cell is weighted by $(d_i + e)/d_i$, about 1.03 for $e = 0.2$ and a typical valence of 6. The feedback therefore acts as a proximal damping of the iterations rather than as a change of the model. On the bar bent into a 90° arc, the shapes converged with $e = 0.2$ and with $e = 0$ (3,000 iterations each) differ by at most 0.33% of the bar's length, and their ARAP distortions by 0.9%.

5.6. Symmetric form and the exact energy

Multiplying the four equations of vertex i in (5.7) by $s_i = (d_i + e)/(\alpha A)$ makes every off-diagonal block equal to $-I$:

$$s_i\,(M_i + \alpha A\,I)\,R_i - \sum_{j\in N(i)} R_j = e\,R_i^{prev}, \qquad s_i = \frac{d_i + e}{\alpha A}. \tag{5.9}$$

This scaling of the rows does not change the solution. In matrix form, (5.9) reads $\mathrm{K\,R} = e\,\mathrm{R}^{prev}$, where K has the diagonal blocks $s_i(M_i + \alpha A\,I)$ and the block $-I$ in positions (i, j) and (j, i) for every edge $\{i, j\}$. The matrix K is symmetric.

Proposition 5.2. *The system* $\mathrm{K\,R} = e\,\mathrm{R}^{prev}$ *is the exact stationarity condition of the strictly convex quadratic energy*

$$\tilde{E}(R) = \sum_{i=1}^{m}(d_i+e)\,R_i^T M_i\,R_i + \alpha A \sum_{\{i,j\}\in\mathcal{E}} \|R_i - R_j\|^2 + \alpha A\,e \sum_{i=1}^{m} \|R_i - R_i^{prev}\|^2, \tag{5.10}$$

where $\mathcal{E}$ *is the set of mesh edges, each counted once. Its unique minimizer is the closed-form solution of* (5.8) *without constraints, and also the fixed point that relaxation of* (5.7) *converges to.*

Proof. Each edge $\{i, j\}$ incident to i contributes $2\alpha A(R_i - R_j)$ to $\partial\tilde{E}/\partial R_i$, so

$$\begin{aligned}
\tfrac{1}{2}\,\frac{\partial \tilde{E}}{\partial R_i} &= (d_i + e)\,M_i\,R_i + \alpha A \sum_{j\in N(i)} (R_i - R_j) + \alpha A\,e\,(R_i - R_i^{prev}) \\
&= (d_i + e)\,M_i\,R_i + \alpha A\,(d_i + e)\,R_i - \alpha A \sum_{j\in N(i)} R_j - \alpha A\,e\,R_i^{prev} \\
&= \alpha A\Big(s_i\,(M_i + \alpha A\,I)\,R_i - \sum_{j\in N(i)} R_j - e\,R_i^{prev}\Big),
\end{aligned}$$

which vanishes exactly when (5.9) holds. The first two terms of $\tilde{E}$ are positive semi-definite quadratic forms and the feedback term is positive definite, so $\tilde{E}$ is strictly convex and its stationary point is its unique minimizer. □

$\tilde{E}$ is the linearized SR-ARAP energy with three differences: the membrane term of cell i is weighted by $d_i + e$, the bending term is the symmetric, unweighted sum over edges, and the feedback appears as an explicit proximal term towards the previous rotations. The closed-form local step is thus not an approximation of relaxation: it is the exact minimizer of $\tilde{E}$, which relaxation can only approach iteratively.

Remark 5.3. The symmetrized gradient (5.5) gives another symmetric formulation, with the weights $w_{ij} + w_{ji}$, whose solution is the exact minimizer of E itself. Formulation (5.9) is the one that matches relaxation of (5.7).

5.7. Rotation constraints and solution

With the rotations of the constrained vertices fixed by (5.6), the rows of the free vertices $\mathcal{F}$ read $\mathrm{K}_{\mathcal{F}\mathcal{F}}\,\mathrm{R}_{\mathcal{F}} + \mathrm{K}_{\mathcal{F}\mathcal{C}}\,\mathrm{R}_{\mathcal{C}}^{const} = e\,\mathrm{R}_{\mathcal{F}}^{prev}$. Since the off-diagonal blocks of K are $-I$, the four equations of a free vertex i become

$$s_i\,(M_i{+}\alpha A\,I)\,R_i{-}\sum_{j\in N(i)\cap\mathcal{F}} R_j = e\,R_i^{prev}{+}\sum_{j\in N(i)\cap\mathcal{C}} R_j^{const},\qquad i\in\mathcal{F}. \tag{5.11}$$

Equivalently, $\mathrm{R}_{\mathcal{F}}$ is the minimizer of $\tilde{E}$ over the free rotations, with the constrained ones fixed to R_i^{const}.

Proposition 5.4. *$\mathrm{K}_{\mathcal{F}\mathcal{F}}$ is symmetric positive definite, with smallest eigenvalue at least e.*

Proof. It is a principal submatrix of the symmetric matrix K. Since M_i is positive semi-definite, $s_i(M_i + \alpha A\,I) \succeq s_i\,\alpha A\,I = (d_i + e)\,I$. For $x \in \mathbb{R}^{4|\mathcal{F}|}$ with blocks x_i, using $2\,x_i \cdot x_j \le \|x_i\|^2 + \|x_j\|^2$ on every edge between free vertices,

$$x^T\mathrm{K}_{\mathcal{F}\mathcal{F}}\,x \ge \sum_{i\in\mathcal{F}}(d_i + e)\,\|x_i\|^2 - \sum_{i\in\mathcal{F}}|N(i)\cap\mathcal{F}|\,\|x_i\|^2 \ge e\,\|x\|^2. \qquad \square$$

Unlike Proposition 4.1, this holds for any constraint set, even an empty one, thanks to the feedback term. System (5.11) can therefore be solved with a sparse Cholesky factorization, and no least-squares formulation is needed. Its sparsity pattern is that of the mesh graph with 4×4 blocks and depends only on the mesh and the constraint set, so the symbolic analysis (the fill-reducing ordering) is computed once. The values, however, change in every iteration, since M_i depends on the deformed positions, so a direct solve costs one numeric factorization per local step, whose cost grows as $n^{1.5}$ on surface meshes. The next section removes that factorization. In either case the solution quaternions R_i are normalized afterwards.

5.8. Solving the rotation system in near-linear time

Write $\mathrm{K}_{\mathcal{F}\mathcal{F}}$ as the sum of a part that changes with the positions and a part that does not:

$$\mathrm{K}_{\mathcal{F}\mathcal{F}} = \mathrm{C}\otimes I_4 + \mathrm{B},\qquad \mathrm{B} = \mathrm{diag}\big(s_i M_i\big)_{i\in\mathcal{F}}, \tag{5.12}$$

where C is the $|\mathcal{F}|\times|\mathcal{F}|$ scalar matrix with diagonal $s_i(\alpha A{+}\eta)$ and the entry -1 for every edge between free vertices, and η is the small diagonal regularization of Section 8. C acts identically on the four quaternion components and depends only on the mesh, the constraint set and the feedback weight; B is block diagonal and positive semi-definite, and carries all the dependence on the deformed positions.

Proposition 5.5. *Let $\mu = \max_{i\in\mathcal{F}} s_i\,\lambda_{\max}(M_i)$ and $d_{\max} = \max_i d_i$. Then*

$$\kappa\big(\mathrm{K}_{\mathcal{F}\mathcal{F}}\big) \le \frac{2d_{\max} + e + \mu}{e}\,(1 + \eta/\alpha A),\qquad \kappa\big((\mathrm{C}\otimes I_4)^{-1}\mathrm{K}_{\mathcal{F}\mathcal{F}}\big) \le 1 + \frac{\mu}{e}.$$

Proof. The diagonal of C is $s_i(\alpha A + \eta) = (d_i + e)(1 + \eta/\alpha A) \geq d_i + e$ and its off-diagonal entries in row i sum in absolute value to at most d_i, so by Gershgorin's theorem the eigenvalues of C lie in $[e, (2d_{\max} + e)(1 + \eta/\alpha A)]$. Since $0 \preceq \mathrm{B} \preceq \mu I$, the eigenvalues of $\mathrm{K}_{\mathcal{FF}}$ lie in $[e, (2d_{\max}+e)(1+\eta/\alpha A)+\mu]$, which gives the first bound. For the second, the generalized Rayleigh quotient is

$$\frac{x^T \mathrm{K}_{\mathcal{FF}}\, x}{x^T (\mathrm{C} \otimes I_4)\, x} = 1 + \frac{x^T \mathrm{B}\, x}{x^T (\mathrm{C} \otimes I_4)\, x} \in \Big[1,\, 1 + \frac{\mu}{e}\Big]. \qquad \square$$

Both bounds are independent of the resolution of the mesh. Moreover μ decreases under refinement: $\lambda_{\max}(M_i) \leq \operatorname{tr} M_i = 4 \sum_j c_{ij} (\|p_{ij}\|^2 + \|q_{ij}\|^2)$ scales with the squared edge length, so for a uniform refinement of a fixed surface with bounded stretching $\mu = O(1/n)$, and the preconditioned condition number tends to 1. We therefore solve (5.11) by the preconditioned conjugate gradient method (PCG) with preconditioner $\mathrm{C} \otimes I_4$, warm-started from the previous rotations. Since the error of PCG decreases by at least $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ per iteration, the number of iterations needed for a given accuracy is bounded independently of the mesh, and decreases as it is refined.

C is factored once per constraint set with a sparse Cholesky factorization, like the Poisson systems of Sections 4 and 6.1. Every PCG iteration then costs one product with $\mathrm{K}_{\mathcal{FF}}$, which is $O(n)$, and one application of the preconditioner: two triangular solves with four right-hand sides, $O(n \log n)$ for the nested-dissection orderings of surface meshes. A local step therefore costs $O(n \log n)$, instead of the $O(n^{1.5})$ of a numeric factorization. We stop PCG at a relative residual of 10^{-12}, where its rotations match those of the direct solve to within 10^{-12} rad, so the method keeps the exactness of the closed form. Without a preconditioner, conjugate gradients also converges in a mesh-independent number of iterations by the first bound of Proposition 5.5, at $O(n)$ per iteration: a linear-time solver, at the price of more iterations.

6. Multiresolution Optimization

To achieve interactive rates, we optimize the SR-ARAP energy on a low-resolution version of the input mesh, the *control mesh*, and then transfer the solution to the full-resolution mesh. The control mesh is obtained by simplifying the input mesh with *half-edge collapses* that minimize the quadric error metric, so that it is a triangulation of a subset of the original vertices. Each control vertex k is associated with a full-resolution vertex $h(k)$: the vertex it was kept from, or equivalently its nearest full-resolution vertex.

Once the optimal deformed shape of the control mesh is found, the optimized rotations are transferred to the full-resolution mesh by harmonic interpolation of quaternions (see Section 6.1). The full-resolution positions are then obtained from Equation (4.1) on the full-resolution mesh, with the interpolated rotations and with the optimized control vertices as positional

constraints, $p_{h(k)} = p_k^{control}$:

$$\sum_{j \in N(i)} c_{ij}\,(p_j - p_i) = \sum_{j \in N(i)} \frac{c_{ij}}{2}\left(R_i\, q_{ij}\, \tilde{R}_i + R_j\, q_{ij}\, \tilde{R}_j\right).$$

Algorithm 1 summarizes one interactive frame.

Algorithm 1 One interactive frame. The Cholesky factorizations of Sections 4 and 6.1, and the symbolic analysis of (5.11), are computed once per constraint set.

1: Set the positional constraints p_i^{const} of the control mesh from the handle transformations.
2: **for** $k = 1$ to K **do**
3: Compute M_i for every control vertex from the current positions (Section 7).
4: Compute R_i^{const}, $i \in \mathcal{C}$, from (5.6); solve (5.11) for R_i, $i \in \mathcal{F}$; normalize all R_i.
5: Solve the global step (4.2) on the control mesh.
6: **end for**
7: Interpolate the control rotations onto the full-resolution mesh (6.4) and normalize them.
8: Solve the global step (4.2) on the full-resolution mesh, constrained to the control positions.

6.1. Harmonic Interpolation of Quaternions

The seminal work of Pinkall and Polthier [9] shows how to interpolate given data over a discrete domain using harmonic maps. Harmonic maps are critical points of the Dirichlet (stretching) energy $\int_\Omega |\nabla f|^2\, dA$, which generates minimal surfaces. The *discrete harmonic energy* of a map f defined on the mesh vertices $\{q_i\}$ has the form

$$E(f) = \sum_{\{i,j\} \in \mathcal{E}} c_{ij}\, \|f(q_j) - f(q_i)\|^2, \tag{6.1}$$

where c_{ij} are the symmetric cotangent weights [9] of the edges. The discrete Laplacian operator can be identified in that energy as

$$\Delta f(q_i) = \sum_{j \in N(i)} c_{ij}\left(f(q_j) - f(q_i)\right). \tag{6.2}$$

Harmonic energy is known to minimize angular distortion, which means that harmonic functions blend boundary conditions smoothly over the domain. Harmonic functions are intrinsic to surfaces and independent of the discretization used to produce the meshes. In a spirit similar to [12], we interpolate quaternions over a mesh with harmonic functions. The harmonic quaternion interpolation over a surface mesh is the solution of the discrete harmonic equation

$$\sum_{j \in N(i)} c_{ij}\,(R_j - R_i) = 0 \tag{6.3}$$

subject to Dirichlet boundary conditions $R_{h(k)} = R_k^{control}$, where $\{R_k^{control}\}$ are the optimized quaternions of the control mesh. This interpolation produces a smooth field of quaternions by "averaging" the boundary conditions gradually over the surface, in effect a linear interpolation of the boundary conditions across the surface. In matrix form,

$$\mathrm{L\,R} = 0 \qquad \text{s.t. } R_{h(k)} = R_k^{control},$$

where L is the full-resolution Laplacian of the global step and R is the $m \times 4$ matrix of quaternion coefficients. Substituting the boundary conditions as in Section 4, with $\mathcal{C} = \{h(k)\}$, gives

$$(-\mathrm{L}_{\mathcal{F}\mathcal{F}})\,\mathrm{R}_{\mathcal{F}} = \mathrm{L}_{\mathcal{F}\mathcal{C}}\,\mathrm{R}_{\mathcal{C}}^{control}, \tag{6.4}$$

which has the same matrix as the full-resolution global step, whose constrained vertices are the same set $\{h(k)\}$. One Cholesky factorization therefore serves both the rotation interpolation, with four right-hand sides, and the full-resolution positions, with three. The interpolated quaternions are not of unit length and must be normalized afterwards. Despite its simplicity, this approach works surprisingly well, is extremely efficient, and provides a natural propagation at no extra cost.

7. The form of M_i

As noted in Section 5, for a unit quaternion R_i the membrane term equals

$$\sum_j c_{ij}\,\|R_i\,q_{ij}\,\tilde{R}_i - p_{ij}\|^2 = \sum_j c_{ij}\,\|R_i\,q_{ij} - p_{ij}\,R_i\|^2.$$

Write $R_i = w_i + B_i$, with scalar part w_i and pure quaternion part B_i. Expanding the quaternion products in terms of the inner and cross products gives

$$R_i\,q_{ij} - p_{ij}\,R_i = -\,d_{ij}\cdot B_i + w_i\,d_{ij} - s_{ij}\times B_i, \qquad d_{ij} = q_{ij} - p_{ij}, \quad s_{ij} = q_{ij} + p_{ij}, \tag{7.1}$$

where the first term is the scalar part and the last two are the vector part. In matrix form,

$$M_{ij}R_i = \begin{bmatrix} 0 & -d_{ij}^T \\ d_{ij} & [s_{ij}]_\times^T \end{bmatrix} \begin{bmatrix} w_i \\ B_i \end{bmatrix} = \begin{bmatrix} -d_{ij}^T B_i \\ w_i\,d_{ij} - s_{ij} \times B_i \end{bmatrix}, \tag{7.2}$$

where d_{ij} and s_{ij} are 3×1 column vectors holding pure quaternion coefficients and M_{ij} is a skew-symmetric 4×4 real matrix, $M_{ij}^T = -M_{ij}$. The quaternion R_i is represented as the 4×1 column vector made of the scalar w_i and the 3×1 column vector B_i. The 3×3 matrix $[s_{ij}]_\times$ is the skew-symmetric cross-product matrix, $[s]_\times\, x = s \times x$.

The membrane term is then the quadratic form $R_i^T M_i\,R_i$ with $M_i = \sum_j c_{ij}\,M_{ij}^T M_{ij}$, as in (5.1). Each product $M_{ij}^T M_{ij}$ is symmetric and positive

semi-definite, and since $c_{ij} \geq 0$ so is M_i: all its eigenvalues are real and non-negative. Using $M_{ij}^T = -M_{ij}$,

$$M_{ij}^T M_{ij} = \begin{bmatrix} \|d_{ij}\|^2 & (s_{ij} \times d_{ij})^T \\ s_{ij} \times d_{ij} & d_{ij} d_{ij}^T - [s_{ij}]_\times^2 \end{bmatrix}. \tag{7.3}$$

7.1. Efficient Computation of M_i

Since $[s]_\times^2 = s\, s^T - \|s\|^2 I$, the product $M_{ij}^T M_{ij}$ has the simple form

$$M_{ij}^T M_{ij} = \begin{bmatrix} \|d_{ij}\|^2 & (s_{ij} \times d_{ij})^T \\ s_{ij} \times d_{ij} & d_{ij} d_{ij}^T - s_{ij} s_{ij}^T + \|s_{ij}\|^2 I_{3\times 3} \end{bmatrix}.$$

Writing it in terms of q_{ij} and p_{ij}, with $\|d_{ij}\|^2 = \|q_{ij}\|^2 + \|p_{ij}\|^2 - 2\, q_{ij}^T p_{ij}$, $s_{ij} \times d_{ij} = -2\, q_{ij} \times p_{ij}$ and $d_{ij} d_{ij}^T - s_{ij} s_{ij}^T = -2\,(q_{ij} p_{ij}^T + p_{ij} q_{ij}^T)$, gives

$$\begin{aligned} M_{ij}^T M_{ij} = & -2 \begin{bmatrix} q_{ij}^T p_{ij} & (q_{ij} \times p_{ij})^T \\ q_{ij} \times p_{ij} & q_{ij} p_{ij}^T + p_{ij} q_{ij}^T - q_{ij}^T p_{ij}\, I_{3\times 3} \end{bmatrix} \\ & + (\|p_{ij}\|^2 + \|q_{ij}\|^2)\, I_{4\times 4}. \end{aligned} \tag{7.4}$$

All terms of (7.4) can be derived from the dyadic tensor $q_{ij} p_{ij}^T$ and the scalar $\|p_{ij}\|^2 + \|q_{ij}\|^2$. Summing over the cell, M_i is obtained from the single 3×3 matrix $S_i = \sum_j c_{ij}\, q_{ij} p_{ij}^T$ and the scalar $\sigma_i = \sum_j c_{ij} (\|p_{ij}\|^2 + \|q_{ij}\|^2)$: the trace of S_i gives $\sum_j c_{ij}\, q_{ij}^T p_{ij}$, its antisymmetric part gives $\sum_j c_{ij}\, q_{ij} \times p_{ij}$, and $S_i + S_i^T$ gives $\sum_j c_{ij} (q_{ij} p_{ij}^T + p_{ij} q_{ij}^T)$. Computing M_i therefore costs one 3×3 outer-product accumulation per edge.

8. Implementation

The author's source code is publicly available on GitHub [7]. The implementation divides Equations (5.7) and (5.6) by two, working with $\frac{1}{2} M_i$ and $\frac{\alpha}{2}$. It uses $\alpha = 0.28$ (that is, $\frac{\alpha}{2} = 0.14$) and $e = 0.2$, and adds $\eta = 2 \cdot 10^{-6}$ to the diagonal of every block $M_i + \alpha A\, I$ for numerical safety, which adds the Tikhonov term $\eta \sum_i (d_i + e)\, \|R_i\|^2$ to (5.10). Each interactive frame performs $K = 3$ local/global iterations on the control mesh. The local step is solved by PCG (Section 5.8), warm-started from the previous rotations and stopped at a relative residual of 10^{-12}; its preconditioner C is factored when the constraints are set and reused afterwards. Convergence is monitored with the energy (3.1), with $w_{ij} = 1/d_i$.

9. Results

Setup. All measurements were taken on an Apple M1 Pro laptop (10 cores, 32 GB), with a port of the reference implementation [7] to Rust that uses the sparse Cholesky factorization of the `faer` library; every factorization and solve runs on a single thread, except that the right-hand sides of a Poisson solve are solved in parallel. The local step is solved by PCG with the factor-once preconditioner (Section 5.8); we call its solution, and that of a direct

TABLE 1. Cost of one interactive frame, in milliseconds: the local step on the control mesh with PCG, and for comparison with a direct Cholesky factorization and with 3 relaxation sweeps; the global step (local and global steps are each performed $K = 3$ times per frame); the harmonic interpolation of the rotations; and the full-resolution positions.

Model	Control	Full	PCG	Direct	Relax. 3	Global	Transfer	Detail
Cactus	502	5,261	1.10	1.59	0.07	0.08	0.51	0.26
Armadillo	502	172,974	0.79	0.47	0.08	0.07	14.84	12.76

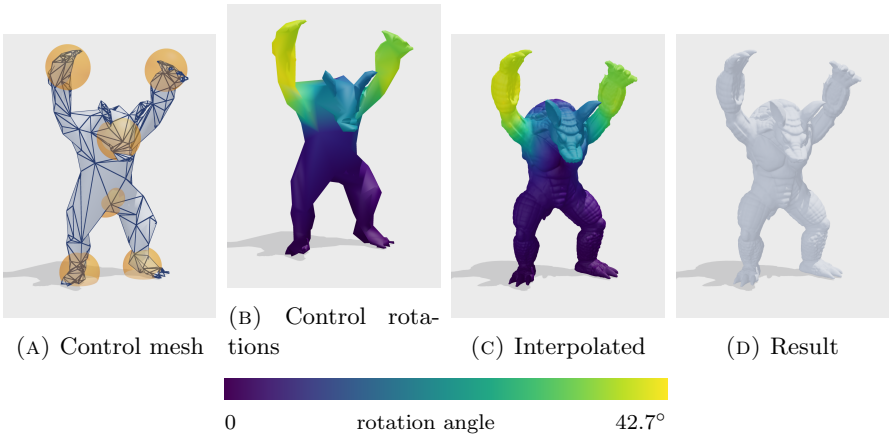
Frame: 4.9 ms (cactus) and 29.1 ms (Armadillo).

sparse Cholesky factorization of $\mathrm{K}_{\mathcal{F}\mathcal{F}}$, the closed form, since the two agree to about 10^{-12} rad. Unless stated otherwise, $\alpha = 0.28$, $e = 0.2$ and $K = 3$ local/global iterations per frame (Section 8). The models are the Armadillo (172,974 vertices) and a cactus (5,261 vertices), each with a 502-vertex control mesh, and a tube, the classic bar benchmark of ARAP papers, generated at resolutions from 530 to 124,010 vertices; its ends are capped by concentric rings so that no vertex has more than 12 neighbors. The relaxation baseline solves exactly the same per-cell equations (5.7): each sweep visits every free cell once and solves its 4×4 system with the current neighbor rotations (block Gauss–Seidel), and the quaternions are normalized after the last sweep. By Proposition 5.2, relaxation and the closed form share the same solution; with $e = 0.2$ we verified that relaxation reaches it to machine precision (10^{-15} rad) on every mesh.

9.1. Interactive multiresolution deformation

Figure 1 shows poses of the Armadillo produced with the multiresolution scheme, and Figure 2 its stages: the rotations found on the control mesh are carried smoothly to the full surface by the harmonic interpolation, with no visible seams at the coarse cells. Table 1 breaks one interactive frame into its stages. With a 502-vertex control mesh, one local step costs 0.8–1.1 ms with PCG (15 to 21 iterations) and 0.5–1.6 ms with a direct factorization; at this size the two are comparable. On the Armadillo the three local steps of a frame take 2.4 ms of 29.1 ms, and the frame is dominated by the two full-resolution solves of Section 6.1, which share one Cholesky factorization; on the cactus the local steps take most of a 4.9 ms frame. In this setting the exact solution of the local step costs little.

The symmetric reduction of Sections 4 and 5.6 matters in practice. In the reference C++ implementation (Eigen), replacing the sparse LU of the row-replaced systems by Cholesky factorizations of the reduced systems lowers the frame time from 18.2 to 6.6 ms on the cactus and from 266 to 182 ms on the Armadillo, with results identical to 10^{-13}; solving the rotation system by PCG instead lowers it further to 6.1 ms on the cactus, with rotations within $2.4 \cdot 10^{-12}$ rad of the LU solution.



(A) Control mesh (B) Control rotations (C) Interpolated (D) Result

FIGURE 2. The multiresolution pipeline on the Armadillo, for the pose of Figure 1(c). (a) The deformed 502-vertex control mesh and its handles. (b) The rotation angle of every control cell after the closed-form local step. (c) The rotation field interpolated harmonically onto the 172,974 vertices of the full mesh. (d) The full-resolution surface reconstructed from the interpolated rotations, with the control vertices as constraints.

9.2. Comparison with SR-ARAP

We compare against the original SR-ARAP local step of Levi and Gotsman [5], run with the same global step, constraints and multiresolution transfer. It keeps exact rotation matrices: each cell's rotation maximizes $\mathrm{tr}(R_i^T S_i)$ with

$$S_i = \sum_{j \in N(i)} c_{ij}\, p_{ij} q_{ij}^T + \alpha' A \sum_{j \in N(i)} (w_{ij} + w_{ji})\, R_j,$$

found by a singular value decomposition (orthogonal Procrustes) with the neighboring rotations fixed, and cells are visited in Gauss–Seidel sweeps. Each update therefore minimizes the total SR-ARAP energy, with bending term $\alpha' A \sum_i \sum_j w_{ij} \|R_i - R_j\|_F^2$, exactly over one cell. We use $w_{ij} = 1/d_i$ and $\alpha' = \alpha/8$, which gives both bending terms the same stiffness for small rotation differences ($\|R_i - R_j\|_F^2 \approx 2\theta^2$ and $\|q_i - q_j\|^2 \approx \theta^2/4$ for rotations θ apart). The two methods do not minimize the same energy, so we compare the resulting shapes with two measures that depend only on the deformed positions: the ARAP distortion $\sum c_{ij} \|R_i q_{ij} - p_{ij}\|^2 / \sum c_{ij} \|q_{ij}\|^2$ of the best rigid fit R_i of every cell, and the RMS angle between the fitted rotations of adjacent cells (roughness).

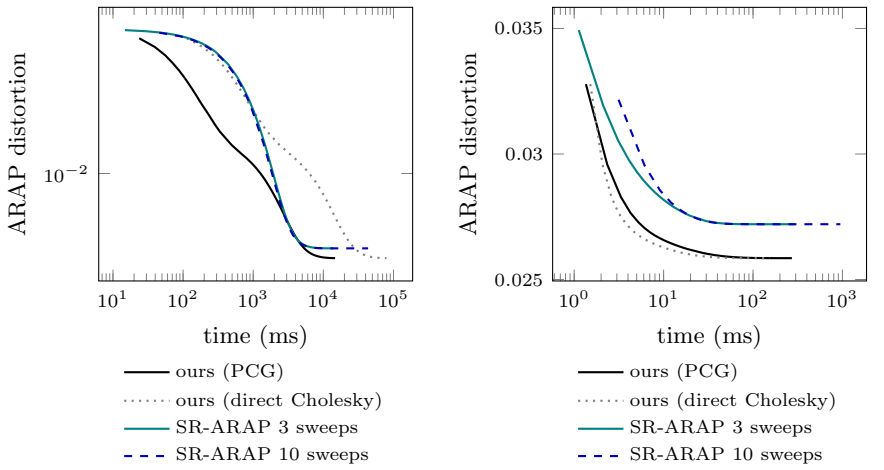
Table 2 and Figure 3 report both measures on the bar bent into a 90° arc and on the Armadillo, against the wall-clock time of the local/global iterations. The closed form converges to shapes with lower distortion and smoother rotations: 12% lower distortion and 1.18° against 1.37° roughness

TABLE 2. Our local step (PCG) against the original SR-ARAP local step after convergence (1,000 iterations on the bar, 300 on the Armadillo). Time is one local step; distortion and roughness are measured on the best rigid fit of every cell; gap is the largest distance to our shape, relative to the half-diagonal.

Model	Local step	Time (ms)	Distortion	Roughness	Gap
Bar, 90° arc	ours (PCG)	13.8	$3.42 \cdot 10^{-3}$	1.18°	–
	SR-ARAP, 3 sweeps	13.1	$3.87 \cdot 10^{-3}$	1.37°	2.6%
	SR-ARAP, 10 sweeps	43.9	$3.87 \cdot 10^{-3}$	1.37°	2.6%
Armadillo	ours (PCG)	0.84	$2.59 \cdot 10^{-2}$	7.56°	–
	SR-ARAP, 3 sweeps	0.90	$2.72 \cdot 10^{-2}$	7.68°	1.6%
	SR-ARAP, 10 sweeps	2.96	$2.72 \cdot 10^{-2}$	7.68°	1.6%

on the bar, and 5% lower distortion on the Armadillo. SR-ARAP with 3 and with 10 sweeps per iteration converges to the same shape, so the difference does not come from the sweep budget. The shapes differ by up to 2.6% of the model's half-diagonal on the bar and 1.6% on the Armadillo; Figure 9 shows where. Our method also gets there sooner. Measured by the wall-clock time to come within 1% of the distortion SR-ARAP converges to, PCG takes 5.3 s on the bar, against 7.2 s for SR-ARAP with 3 or with 10 sweeps per iteration (1.4 times faster), and 5.2 ms on the Armadillo, against 24 ms for SR-ARAP with 10 sweeps and 28–30 ms with 1 or 3 (4.6 to 5.7 times faster); it then continues to the lower distortion that SR-ARAP does not reach. Counted in iterations rather than time, the picture is more nuanced: on the Armadillo the closed form is ahead from the fifth iteration on, but on the bar, which undergoes much larger rotations, it leads for the first 30 iterations, after which SR-ARAP with 10 sweeps approaches its final shape in fewer iterations (within 1% after 160, against 350), each of which costs three times as much. With a direct factorization of $\mathrm{K}_{\mathcal{F}\mathcal{F}}$ in every iteration instead of PCG, the bar would take 27.8 s: the speed-up over SR-ARAP relies on the iterative solver of Section 5.8. We attribute the slower start to the feedback term, which pulls every rotation towards its previous value: it provides temporal coherence and conditioning, at the price of a slower outer loop. The lower distortion does not come from the feedback: without it ($e = 0$) the closed form converges to an even lower distortion on the bar, $3.38 \cdot 10^{-3}$.

An SR-ARAP sweep costs one 3×3 SVD per cell, about ten times a sweep of the linearized relaxation (4.4 against 0.4 ms on the bar). A PCG local step costs about as much as three SR-ARAP sweeps (13.8 against 13.2 ms on the bar, 0.84 against 0.91 ms on the Armadillo's control mesh), but it is exact. Inexact relaxation of our linearized system with 3 sweeps per iteration is faster still (0.7 s on the bar), but its rotations are not exact.



(A) Bar, 90° arc (B) Armadillo, 502-vertex control mesh

FIGURE 3. ARAP distortion of the shape against the wall-clock time of the local/global iterations, with our local step (solved by PCG, or by a direct factorization in every iteration) and with the original SR-ARAP local step.

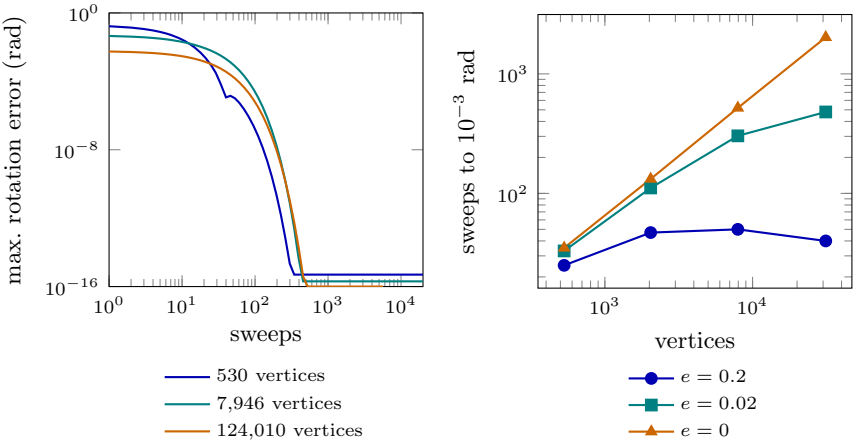
9.3. Closed form and relaxation

Figure 4(a) shows how far relaxation is from the closed-form solution after each sweep, in the hardest case for relaxation: the first local step after the handle of a tube is bent by 60°, with all rotations still at rest. With the budget of two or three sweeps that interactive SR-ARAP implementations use, the rotations are off by up to 0.11 rad (6.6°) on the coarsest tube and $5 \cdot 10^{-3}$ rad on the finest; the closed form has no such error.

The number of sweeps needed to reach a given accuracy, however, does not grow with the mesh: 25 to 50 sweeps reach 10^{-3} rad at every resolution. The reason is the feedback term. By Proposition 5.4, the eigenvalues of $\mathrm{K}_{\mathcal{FF}}$ are bounded below by e, independently of the mesh, so the system is well conditioned and relaxation contracts at a mesh-independent rate. Figure 4(b) confirms this: with $e = 0.2$ the sweep count stays flat, while with weaker feedback it grows, and without feedback ($e = 0$, the plain smooth-rotation system) it grows linearly with the number of vertices, from 35 sweeps at 530 vertices to 2,022 at 31,274.

9.4. Solving the rotation system: direct and iterative

Table 3 and Figure 5 compare three ways of computing the same exact local step on the bent tube, in the cold start of Figure 4: a direct sparse Cholesky factorization of $\mathrm{K}_{\mathcal{FF}}$ in every local step (with the symbolic analysis reused), conjugate gradients without a preconditioner, and PCG with the factor-once preconditioner C of Section 5.8. Both iterative solvers start from the previous



(A) Relaxation error, cold start (B) Effect of the feedback weight e

FIGURE 4. Relaxation converges to the closed-form solution (a), but with a budget of a few sweeps it leaves rotation errors of several degrees. The number of sweeps it needs is independent of the mesh with the feedback term and grows linearly without it (b). Tube bent by 60°, first local step after the handle moves.

rotations and are run until their rotations are within 10^{-12} rad of the direct solution.

The direct factorization grows superlinearly, roughly as $n^{1.5}$: from 1.8 ms at 530 vertices to 84 ms at 7,946 and 5.7 s at 124,010. Conjugate gradients needs 59 to 77 iterations at every resolution, as the first bound of Proposition 5.5 predicts, and its cost grows linearly, as $n^{0.94}$, reaching 0.31 s at 124,010 vertices. PCG needs even fewer iterations as the mesh is refined, 18 at 530 vertices and 5 at 124,010, as the second bound predicts; with each iteration costing one back-substitution, it solves the local step in 1.1 ms at 530 vertices and 0.11 s at 124,010, 1.7 to 49 times faster than the direct factorization. The factorization of C itself takes 0.4 to 110 ms and is done once per constraint set. From about 8,000 vertices on, PCG reaches the exact solution in the time relaxation needs for an accuracy of 10^{-3} rad (Figure 5); on the smallest tubes relaxation to that accuracy remains a few times cheaper; preconditioning with the inverses of the 4×4 diagonal blocks (block Jacobi) brings no improvement over unpreconditioned conjugate gradients.

With feedback, relaxation also converges in a mesh-independent number of sweeps, but only to the accuracy it is given time for. Without feedback ($e = 0$) its sweep count grows linearly with the mesh (Figure 4(b)), while the second bound of Proposition 5.5 no longer holds; the first bound degenerates as $e \to 0$ as well, and a direct factorization, which needs no bound on the conditioning, remains available.

TABLE 3. Time of one exact local step on the tube (cold start), in milliseconds, with the number of iterations of the iterative solvers. Iterative solvers stop when their rotations are within 10^{-12} rad of the direct solution; their times include the assembly of the system. The factorization of the PCG preconditioner, done once per constraint set, is listed separately.

Vertices	Direct Cholesky	CG	PCG	Factor C (once)
530	1.8	1.8 (59)	1.1 (18)	0.4
2,042	12.0	4.4 (72)	2.0 (12)	1.1
7,946	84.1	19.0 (75)	7.0 (8)	7.5
31,274	657.0	80.0 (77)	27.3 (6)	20.1
124,010	5,677	308.6 (74)	114.9 (5)	109.9

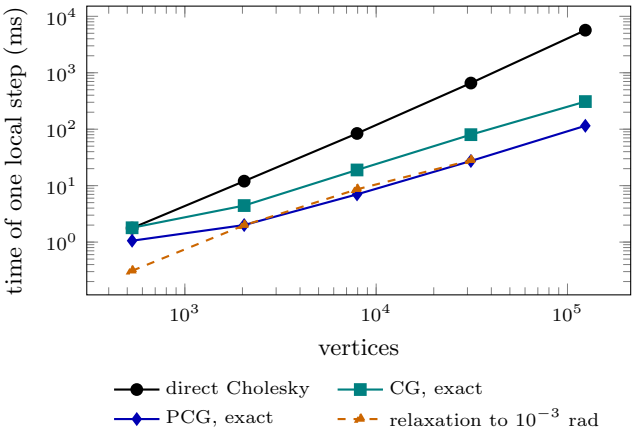
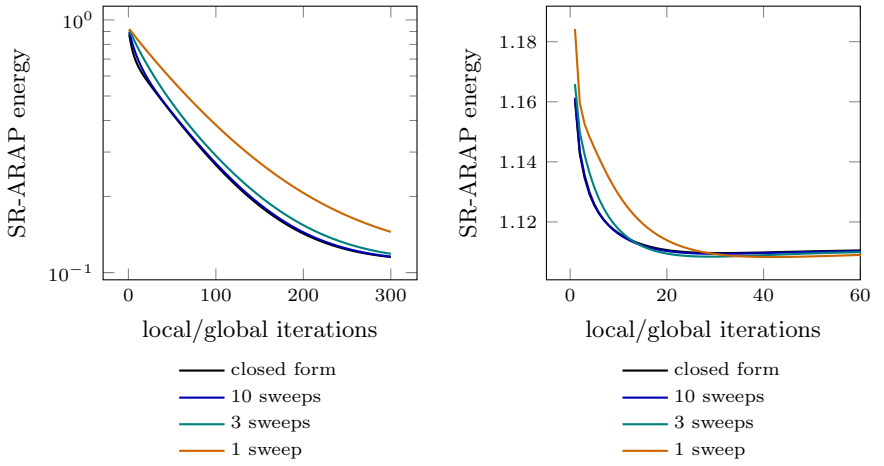


FIGURE 5. Time of one local step on the tube against its number of vertices. The direct factorization grows as $n^{1.5}$; conjugate gradients, which needs a mesh-independent number of iterations, grows linearly; PCG with the factor-once preconditioner needs fewer iterations as the mesh grows, is the fastest exact solver at every size, and on the larger tubes is as fast as inexact relaxation.

9.5. Effect on the optimization

Figure 6 follows the SR-ARAP energy (3.1) through 300 local/global iterations on the bent tube (7,946 vertices) and on the Armadillo. On the tube, the closed form gives the lowest energy at every iteration: after 30 iterations the energy is 0.524 with the closed form, against 0.531, 0.594 and 0.693 with 10, 3 and 1 relaxation sweeps per iteration; after 300 iterations relaxation with one sweep is still 26% above the closed form. On the 502-vertex control mesh



(A) Tube, 7,946 vertices (B) Armadillo, 502-vertex control mesh

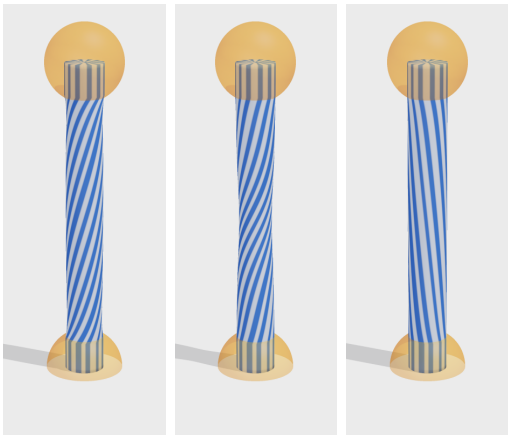
FIGURE 6. SR-ARAP energy per local/global iteration, with the closed-form local step and with relaxation limited to 1, 3 or 10 sweeps per iteration.

of the Armadillo, the closed form leads during the first ten iterations, after which all methods agree to within 0.1% (0.04% after 300 iterations). Since the closed form minimizes $\tilde{E}$ rather than E (Proposition 5.2), relaxation with a few sweeps can end slightly below it in E.

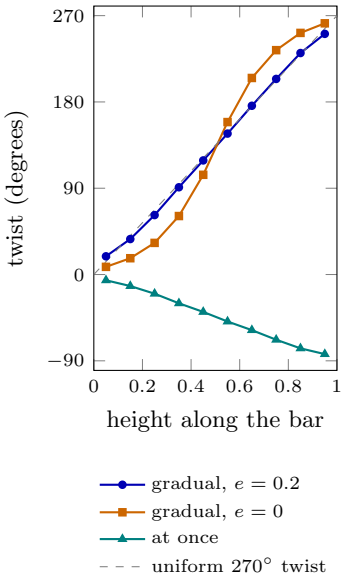
9.6. Feedback and large twists

To see what the feedback contributes for large rotations, we twist the top of the bar by 270°. A handle rotated by 270° places its vertices exactly where a rotation by −90° would, so a single solve cannot tell the two apart: set at once, the solver produces the −90° twist (Figure 7(c)), the right answer to that static problem. In interactive use, however, the handle reaches 270° gradually. We turn it by 3° per step, with 20 local/global iterations per step, and then hold it at 270° for another 1,000 iterations, with and without feedback.

With feedback ($e = 0.2$) the twist follows the handle all the way to 270° and spreads evenly along the bar: its angle grows almost linearly with height, from 19° in the lowest tenth of the bar to 251° in the highest (Figure 7(a) and (d)). Without feedback ($e = 0$) the rotations also follow the 270° branch, since they are warm-started from the previous step, but the twist bunches in the middle of the bar: between 35% and 65% of the height it grows by 144°, against 85° with feedback and 81° for a uniform twist. The feedback thus does more than keep the branch: by anchoring every cell to its previous rotation, it lets the twist distribute along the bar as the handle turns, instead of concentrating where the rotations change fastest.



(A) $e = 0.2$ (B) $e = 0$ (C) At once

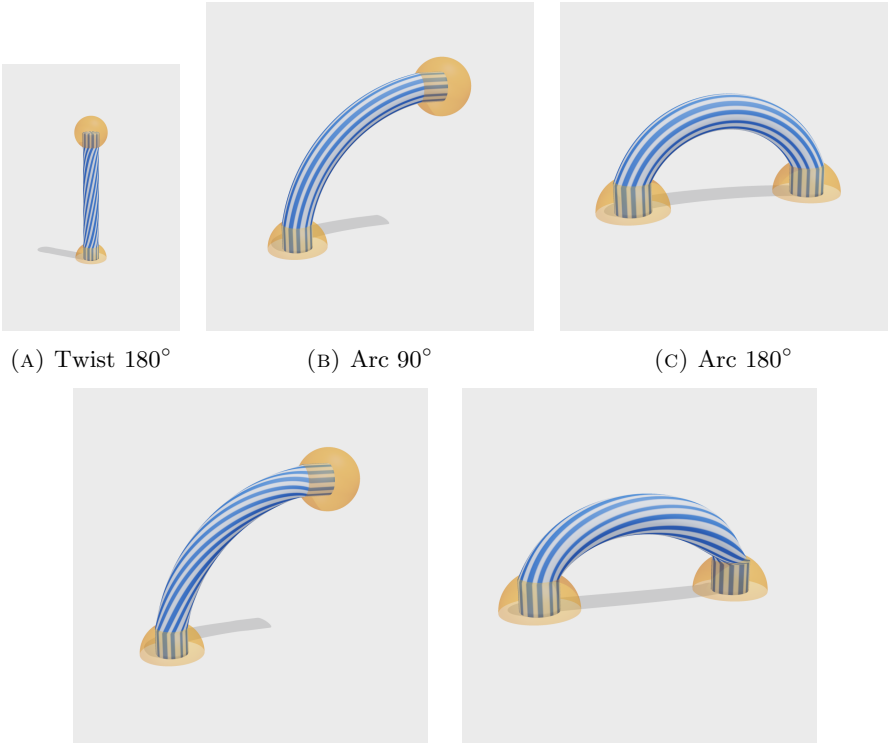


(D) Twist by height

FIGURE 7. A 270° twist of the bar (7,946 vertices), with the closed-form local step. (a), (b) The top handle is turned gradually, by 3° per step with 20 iterations per step, then held for 1,000 iterations, with feedback ($e = 0.2$) and without it ($e = 0$). (c) The handle is set to 270° at once, which places its vertices where −90° would. (d) The mean rotation angle about the bar's axis of the control cells in each tenth of the bar's height.

9.7. Gallery

Figure 8 shows the bar benchmark twisted by 180°, bent into circular arcs of 90° and 180°, and twisted by 180° while following each arc. In the 180° arc the two ends of the bar meet the ground side by side, and the cross-section stays round along the whole arc. The stripes show that the twist is distributed evenly along the bar, as the smooth-rotation term intends: the rotation angle of the cells grows linearly with height, from 11° at the bottom to 162° just below the top handle. Large rotations need many local/global iterations, however: the 180° twist takes about 2,000 iterations to spread along the bar, and after 600 it is still concentrated under the top handle. The closed form makes every local step exact, but it does not shorten the outer loop. Figure 9 compares the 180° arc with the shape the original SR-ARAP local step converges to; they differ by at most 1.6% of the bar length, around the top of the arc. Figure 10 deforms the cactus with two handles. Bending the trunk along a 120° arc folds it over until its top points sideways, with the



(A) Twist 180° (B) Arc 90° (C) Arc 180°

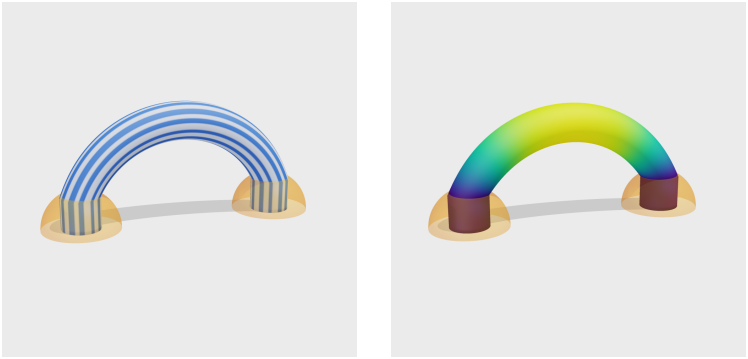
(D) Arc 90°, twist 180° (E) Arc 180°, twist 180°

FIGURE 8. The bar benchmark (7,946 vertices) after 2,000 local/global iterations with the closed-form local step. The bottom handle is fixed. The top handle is twisted about the bar's axis, moved along a circular arc while turning by the arc angle, or both.

arms following rigidly; moving the top handle by two thirds of the trunk's length without rotating it bends the trunk into an S-curve, and the top keeps its orientation.

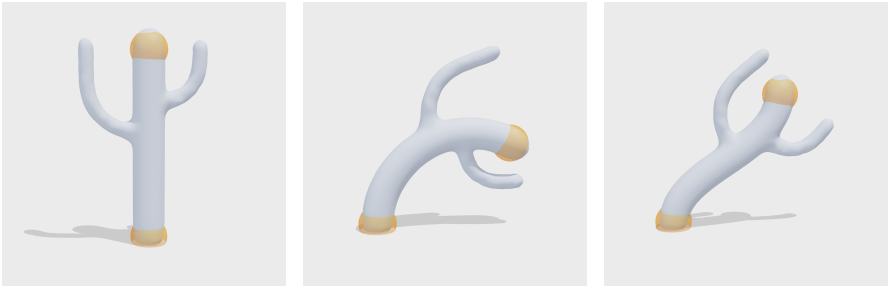
10. Conclusion

We have shown that the local step of SR-ARAP, when rotations are linearized as quaternions, can be solved exactly rather than by relaxation. Scaling the rows of the coupled per-cell equations makes them a single sparse, symmetric positive definite system, and that system is the exact stationarity condition of a quadratic energy, $\tilde{E}$, which differs from the linearized SR-ARAP energy by a valence weighting of the membrane term and an explicit proximal feedback term. The closed form is therefore not an approximation of relaxation: it is



(A) Closed form (B) SR-ARAP, 3 sweeps

FIGURE 9. The 180° arc with the closed-form local step and with the original SR-ARAP local step, both after 2,000 iterations. The colors show the distance between the two shapes.



(A) Rest (B) Arc 120° (C) Moved only

FIGURE 10. The cactus (5,261 vertices, 502-vertex control mesh) with its base fixed. (b) The trunk bent along a 120° arc: the top handle turns by 120° and moves to the end of the arc. (c) The top handle moved by two thirds of the trunk's length, sideways and down, without rotating it.

the minimizer that relaxation converges to. The same reduction makes every Poisson solve of the method symmetric positive definite, so the whole pipeline runs on sparse Cholesky factorizations without least-squares formulations.

The rotation system is also well conditioned. The feedback term bounds its smallest eigenvalue by the feedback weight, and its position-dependent part is small, of order $1/n$ relative to the rest; preconditioning with a Cholesky factorization of the position-independent part, factored once per set of constraints, bounds the condition number by $1 + \mu/e$, a constant that shrinks

as the mesh is refined. Conjugate gradients with this preconditioner reach the exact rotations, to 10^{-12} rad, in 5 to 20 iterations, each costing a sparse product and one back-substitution. The exact local step therefore costs nearly linear time: on the largest tube it is 49 times faster than a direct factorization and as fast as a few relaxation sweeps, which are not exact.

The experiments support three further conclusions. First, exactness matters at the iteration budgets used in practice: two or three relaxation sweeps leave rotation errors of several degrees, and limited sweeps slow the decrease of the energy. Second, the converged shapes are of high quality: compared with the original SR-ARAP local step they have 5–12% lower ARAP distortion and smoother rotations, and they reach the distortion at which SR-ARAP stops 1.4 times sooner on a strongly bent bar and 4.6 times sooner on the Armadillo. Third, in the multiresolution setting the exact local step is a small part of an interactive frame on a 173,000-vertex model, 0.8 ms on its 502-vertex control mesh.

The feedback term is part of the contribution. It gives temporal coherence while editing, favors the branch that large rotations are following, and bounds the smallest eigenvalue of the rotation system for any mesh and constraint set, while changing the converged shape by only a fraction of a percent. Its cost is a slower outer loop for very large rotations: on the bent bar the original SR-ARAP approaches its final shape in fewer local/global iterations, although it stops at a higher distortion, and our method is faster in wall-clock time only because each of its iterations is cheaper. Making the outer loop converge in fewer iterations, for example by adapting the feedback weight to the size of the rotation updates, is a natural direction for future work, as is an extension of the derivation to volumetric meshes.

References

[1] I. Chao, U. Pinkall, P. Sanan, and P. Schröder. A simple geometric model for elastic deformations. In *ACM SIGGRAPH 2010 Papers*, SIGGRAPH '10, New York, NY, USA, 2010. Association for Computing Machinery.

[2] S.-Y. Chen, L. Gao, Y.-K. Lai, and S. Xia. Rigidity controllable as-rigid-as-possible shape deformation. *Graphical Models*, 91:13 – 21, 2017.

[3] T. Jiang, K. Qian, S. Liu, J. Wang, X. Yang, and J. Zhang. Consistent as-similar-as-possible non-isometric surface registration. *The Visual Computer*, 33:891–901, 2017.

[4] Y. Le Vaou, J.-C. Léon, S. Hahmann, S. Masfrand, and M. Mika. As-stiff-as-needed surface deformationcombining arap energy with an anisotropic material. *Computer-Aided Design*, 121:102803, 2020.

[5] Z. Levi and C. Gotsman. Smooth rotation enhanced as-rigid-as-possible mesh animation. *IEEE transactions on visualization and computer graphics*, 21(2):264—277, February 2015.

[6] M. C. Lopez Belon. Applications of conformal geometric algebra in mesh deformation. In *Graphics, Patterns and Images (SIBGRAPI), 2013 26th SIBGRAPI - Conference on*, pages 39–46, Aug 2013.

[7] M. C. Lopez Belon. Project on github. Available at https://github.com/mauriciocele/arap-sr-linearized, 2020.

[8] A. Morsucci, M. Centin, and A. Signoroni. Fast Centroidal Deformation for Large Mesh Models. In M. Livesu, G. Pintore, and A. Signoroni, editors, *Smart Tools and Apps for Graphics - Eurographics Italian Chapter Conference*. The Eurographics Association, 2018.

[9] U. Pinkall and K. Polthier. Computing discrete minimal surfaces and their conjugates. *Experimental Mathematics*, 2:15–36, 1993.

[10] O. Sorkine and M. Alexa. As-rigid-as-possible surface modeling. In *Proceedings of the fifth Eurographics symposium on Geometry processing*, SGP '07, pages 109–116, Aire-la-Ville, Switzerland, Switzerland, 2007. Eurographics Association.

[11] J. Wu, M. Lopez, M. Liu, and Y. Zhu. Linear geometric algebra rotor estimator for efficient mesh deformation. *IET Cyber-systems and Robotics*, 2(2):88–95, 2020.

[12] R. Zayer, C. Rössl, Z. Karni, and H. peter Seidel. Harmonic guidance for surface deformation. In *In Proc. of Eurographics 05*, pages 601–609, 2005.

Mauricio Cele Lopez Belon
Madrid, Spain
e-mail: mclopez@outlook.com